Diamond Framework:

Reputation-Weighted Validator Selection via Verifiable Non-Transferable Reputation

Ilich Blanco*

September 2026

Abstract

The Diamond Framework addresses the structural tension in Proof-of-Stake between security and accessibility. High collateral requirements deter honest but capital-constrained validators; low requirements expose the network to Sybil attacks. The framework replaces a fraction of the financial stake with verifiable, non-transferable reputation.

The paper makes four contributions: a cryptoeconomic protocol in which financial stake and reputational collateral are partial substitutes; a numerical verification suite that pairs every quantitative claim with a reproducible script; a falsification log documenting eighteen claims that were made, tested, and corrected during development; and a set of design decisions with explicit boundaries, stating where the protocol cannot deliver a guarantee.

The protocol is verified numerically for the reputation dynamics, the tensor representation, the zero-knowledge threshold proof, the collusion detection, the payment rule, the data layer, and the bootstrap transition. EVM precompile costs are measured locally with Foundry. Two benchmarks are deferred: the recursive SNARK proving time and the Teleporter latency, both requiring infrastructure not available in this paper.

Contents

1 Introduction **4**
- 1.1 The Validator Trust Gap . . . 4
- 1.2 Reputation as Partial Collateral . . . 4
- 1.3 Contributions . . . 4
- 1.4 Paper Structure . . . 5
- 1.5 Reproducibility . . . 5

2 Reputation Dynamics **6**
- 2.1 The CIR Process . . . 6
- 2.2 Domain Validity Under Jumps . . . 6
- 2.3 The Whitewashing Bound . . . 6
- 2.4 Stationary Distribution . . . 7
- 2.5 Integro-Differential Formulation . . . 8
- 2.6 Summary of the Section . . . 8

3 Tensor Compression **8**
- 3.1 Tensor Representation of Reputation . . . 8
- 3.2 Tucker Decomposition . . . 9
- 3.3 Reconstruction Error . . . 9
- 3.4 Error Versus Rank . . . 10

*Andromeda Computer, ilichblanco@andromedacomputer.net

3.5 Tucker Is Not in the Security Path . . . 10
3.6 Summary of the Section . . . 10

4 Zero-Knowledge Threshold Proof **10**
4.1 Circuit Specification . . . 10
4.2 Soundness Failure . . . 11
4.3 Repair Options . . . 11
4.4 Recursive Verification . . . 12
4.5 Estimates and Their Provenance . . . 12
4.6 Summary of the Section . . . 13

5 Collusion Detection **13**
5.1 The Collaboration Graph . . . 13
5.2 The Normalization Problem . . . 13
5.3 The Modularity Matrix . . . 14
5.4 Null Distribution and Calibration . . . 14
5.5 Detection Power . . . 15
5.6 The Detection Boundary . . . 16
5.7 Limitations . . . 16
5.8 Summary of the Section . . . 16

6 Payment Rule **17**
6.1 The Revelation Problem . . . 17
6.2 Verified Reputation . . . 17
6.3 Private Cost of Capital . . . 17
6.4 The Uniform Posted Price . . . 17
6.5 Individual Rationality . . . 18
6.6 Sensitivity . . . 18
6.7 Incentive Compatibility . . . 18
6.8 Summary of the Section . . . 19

7 Data Layer **19**
7.1 Ingestion of Consensus Events . . . 19
7.2 Transport and Freshness . . . 19
7.3 The Fraud Proof Mechanism . . . 19
7.4 Numerical Verification . . . 20
7.5 Slashing and Integrity . . . 20
7.6 Freshness as Second Line of Defense . . . 21
7.7 Summary of the Section . . . 21

8 Bootstrap Dynamics **21**
8.1 The Genesis Problem . . . 21
8.2 Reputation Models . . . 21
8.3 Bootstrap Simulation . . . 22
8.4 Results . . . 22
8.5 Comparison of Policies . . . 23
8.6 Recommendation . . . 23
8.7 Summary of the Section . . . 23

9 Evaluation **23**
9.1 Scope . . . 23
9.2 Reproducibility . . . 24
9.3 On-Chain Gas Costs . . . 24
9.4 Stability of Conclusions . . . 24

9.5 Deferred Benchmarks . . . 25
9.6 Summary of the Section . . . 25

10 Limitations **26**
10.1 Scope . . . 26
10.2 Adversary Model . . . 26
10.3 Collusion Detection Boundary . . . 26
10.4 Collusion by Bipartite Polarization . . . 26
10.5 Reputation Model Selection . . . 27
10.6 Community Structure and Tucker Compression . . . 27
10.7 Calibration on Synthetic Graphs . . . 27
10.8 Bootstrap Assumptions . . . 27
10.9 What the Protocol Does Not Claim . . . 28
10.10Summary of the Section . . . 28

11 Conclusion **28**
11.1 Summary . . . 28
11.2 Methodology . . . 29
11.3 Contributions . . . 29
11.4 Open Problems . . . 29
11.5 Closing Remark . . . 30
11.6 Summary of the Section . . . 30

1 Introduction

1.1 The Validator Trust Gap

Proof-of-stake networks secure their consensus by requiring validators to lock financial collateral. Slashing conditions make misbehavior costly, and the resulting cryptoeconomic security guarantees are well formalized. This mechanism has proven effective. It has also produced a structural tension.

High stake requirements deter honest but capital-constrained participants, concentrating validation power in the hands of large token holders. Low requirements expose the network to Sybil attacks and near-zero-cost misbehavior. The tension is not a design flaw of any particular protocol; it is a consequence of using a single instrument—financial capital—to serve two purposes simultaneously: deterrence and admission.

In proof-of-work networks, physical capacity constraints (hardware, energy, cooling) act as a natural brake on centralization. A miner with a technological advantage cannot expand indefinitely without facing diminishing returns on capacity. In proof-of-stake networks the collateral is purely financial and does not face an equivalent physical brake. The resulting concentration dynamics can be self-reinforcing.

1.2 Reputation as Partial Collateral

The Diamond Framework introduces a second instrument to the admission decision: verifiable, non-transferable reputation. A validator with a long history of contributions faces a large opportunity cost if it misbehaves, because destruction of its reputation eliminates future income streams. This opportunity cost can serve as partial collateral, reducing the financial barrier for honest participants without degrading the Cost of Attack that secures the network.

Making this intuition rigorous requires solving several problems:

1. **Stochastic modeling.** Reputation evolves continuously and is subject to abrupt downward shocks. The dynamics must remain in their positive domain under all admissible shocks.

2. **Verifiability.** Reputation claims must be cryptographically verifiable without revealing the underlying tensor and without relying on a centralized oracle.

3. **Collusion resistance.** Validators may coordinate to inflate each other's reputation. The detection mechanism must be robust to heavy-tailed interaction graphs and must have a stated boundary below which detection is information-theoretically impossible.

4. **Incentive compatibility.** The mechanism must ensure validators report their true reputation, or—if reputation is publicly verified—must make the reporting question vacuous.

5. **Data integrity.** The reputation signal is derived from consensus events on source chains. The transport of these events must be verifiable, and fraud must be detectable and deterred.

The Diamond Framework addresses all five.

1.3 Contributions

The paper makes four contributions.

A cryptoeconomic protocol. We present a validator admission rule in which financial stake and reputational collateral are partial substitutes. The rule is $\mathrm{CoA}(i) = S_f(i) + C_R(T(i)) \geq \Theta(t)$, where $C_R(T)$ is the reputational collateral derived from a verified reputation tensor T. The protocol includes the reputation dynamics, the collusion test, the payment rule, the data layer, and the bootstrap simulation.

A numerical verification suite. Every quantitative claim in the paper is paired with a script that reproduces it. The scripts are self-contained, depend only on NumPy and SciPy, and declare their

seeds and sample sizes. The suite covers stochastic simulation, spectral calibration, game-theoretic equilibrium, and EVM gas benchmarks. Claims without scripts are labeled as conjectures.

A falsification log. During development, eighteen quantitative claims were made, tested, and either confirmed or corrected. The corrections include a fabricated simulation figure, a mismeasurement of a gas cost, a factor of two in an integral, a rounding rule that shifted a phase transition, and an unverified projection of a multidimensional type. The log is part of the paper's record, not a supplement to it.

Design decisions with explicit boundaries. Where the protocol cannot deliver a guarantee—for adversaries that are not rational, for collusion below the Kesten–Stigum boundary, for real reputation data that is not available—the paper states the boundary and does not claim the guarantee. The limitations section separates methodological limits (the numerical methods could be improved) from design limits (the protocol does not solve these problems even with perfect methods).

1.4 Paper Structure

Section 2 presents the reputation dynamics: a CIR diffusion with multiplicative downward jumps, the positivity proof, and the exact mean first-passage time.

Section 3 introduces the tensor representation and the Tucker compression, and shows that the compression fails for block structure while remaining useful for low-rank tensors.

Section 4 specifies the zero-knowledge threshold proof. The original circuit is unsound; the repair uses a Pedersen commitment with Bulletproofs and recursive SNARK composition. The EVM precompile costs are measured locally.

Section 5 develops the collusion detection. The modularity matrix $B = W - dd^\top/(2m)$ replaces the normalized adjacency matrix A_{sym}. The null quantile is calibrated by Monte Carlo, and the detection boundary is characterized.

Section 6 specifies the payment rule: a saturated uniform posted price. Incentive Compatibility is tautological because reputation is verified on-chain and the allocation does not depend on any private type.

Section 7 describes the data layer: the ingestion of consensus events, the transport via state inclusion proofs, and the fraud proof mechanism with its audit equilibrium.

Section 8 presents the bootstrap simulation under the net_gain reputation model, with the comparison of three policies for the transition from pure proof-of-stake to hybrid.

Section 9 evaluates reproducibility, on-chain gas costs, and the stability of the conclusions under parameter variation.

Section 10 states the limitations of the design and of the numerical verification, separated by category.

Section 11 concludes.

1.5 Reproducibility

The paper is accompanied by a public repository containing:

- The verification scripts (`verification/`).
- The EVM benchmarks (`integrations/avalanche/benchmarks/`).
- The falsification log (`docs/audit/numerical-verification.md`).
- The paper source (`docs/paper/sections/`).

Every quantitative claim in the paper can be traced to a script, a benchmark, or a log entry. The commands to reproduce the claims are listed in `verification/README.md`. Running the full suite takes approximately ten minutes on a single core.

2 Reputation Dynamics

2.1 The CIR Process

Validator reputation is modeled as a mean-reverting stochastic process with abrupt downward shocks. In its continuous form, the process follows the Cox–Ingersoll–Ross (CIR) diffusion:

$$dR(t) = \alpha(\bar{R} - R(t))\,dt + \sigma_0\sqrt{R(t)}\,dW(t) \tag{2.1}$$

with $\alpha > 0$ the mean-reversion speed, $\bar{R} > 0$ the long-run mean, and $\sigma_0 > 0$ the diffusion coefficient. The process is strictly positive under the Feller condition $2\alpha\bar{R} \geq \sigma_0^2$, which we assume throughout.

Slashing events are modeled as multiplicative downward jumps:

$$R(t) \mapsto R(t^-) \cdot y, \quad y \in (0, 1] \tag{2.2}$$

where y is drawn from a Lévy measure ν supported on $(0, 1]$. The choice of multiplicative rather than additive jumps is not cosmetic; it is required for the process to remain in its domain.

2.2 Domain Validity Under Jumps

The original formulation of the Diamond Framework used additive jumps of the form:

$$dR(t) = \alpha(\bar{R} - R(t))\,dt + \sigma_0\sqrt{R(t)}\,dW(t) + \int J(z)\tilde{N}(dt, dz) \tag{2.3}$$

with $J(z) < 0$. This specification is not well posed. Additive negative jumps can push R below zero regardless of the Feller condition, because the Feller condition constrains the diffusion, not the jump component.

We quantify this failure by Monte Carlo simulation. With parameters $\alpha = 0.5$, $\bar{R} = 1.0$, $\sigma_0 = 0.8$ (Feller satisfied: $2\alpha\bar{R} = 1.0 \geq \sigma_0^2 = 0.64$), jump intensity $\lambda = 0.8$, mean jump magnitude 0.15, $R(0) = 0.3$, and $T = 5$, the additive-jump specification produces:

Specification	Fraction with $R(t) < 0$
Additive jumps	40.58%
Multiplicative jumps	0.0000%

over 100,000 simulated paths with $dt = 0.001$ [`verification/stochastics/sim_positivity.py`].

The failure rate of 40.58% is not a tail event. With the specified parameters, two in five trajectories violate the domain within five time units. The multiplicative specification resolves this by construction: a jump of the form $R \mapsto R \cdot y$ with $y \in (0, 1]$ preserves positivity pathwise, and the CIR component remains positive under Feller.

2.3 The Whitewashing Bound

For a validator whose reputation has been reduced to near zero, the expected time to recover to a target level R_0 is governed by the first-passage time of the CIR diffusion. Under the scale function $s'(x) = x^{-\kappa}\exp(\beta x)$ with $\kappa = 2\alpha\bar{R}/\sigma_0^2$ and $\beta = 2\alpha/\sigma_0^2$, the classical expression for the expected first-passage time is:

$$\mathbb{E}[\tau_{R_0}] = 2\int_0^{R_0} s'(z)M(z)\,dz \tag{2.4}$$

where $M(z) = \int_0^z m'(y)\,dy$ and $m'(y) = 1/(\sigma^2(y)s'(y))$ is the speed measure density.

We evaluate (2.4) by numerical quadrature in the well-conditioned form

$$\mathbb{E}[\tau_{R_0}] = 2\int_0^{R_0} g(y)\,dy, \quad g(y) = \frac{1}{\sigma_0^2}\int_1^{R_0/y} s^{-\kappa}\exp(\beta y(s-1))\,ds \tag{2.5}$$

which avoids the catastrophic cancellation that arises from computing $s(R_0) - s(y)$ directly when $\kappa > 1$ [`verification/stochastics/mfpt_bound_explicit.py`].

For the two parameter regimes of interest:

Parameters	Exact MFPT	Bound with $C^* = 1$	Implicit C^*
$\alpha = 0.5$, $\sigma_0 = 0.8$	11.2203	19.965	0.562
$\alpha = 2.0$, $\sigma_0 = 0.5$	443.7374	1.415×10^{10}	3.14×10^{-8}

The analytical bound of the original formulation, $C^* \cdot R_0^{1-\kappa}\exp(\beta R_0)$, requires specifying C^*. With $C^* = 1$ the bound exceeds the exact MFPT in both regimes: by a factor of 1.78 in the first, and by 3.19×10^7 (7.5 orders of magnitude) in the second. $C^* = 1$ **is not a valid lower bound for the CIR first-passage time.** The implicit C^* values (0.562 and 3.14×10^{-8}) are the largest constants for which the bound holds.

We verified the numerical values by two independent methods. The first is direct Euler simulation of the CIR [`verification/stochastics/dt_convergence.py`]. The simulated MFPT converges monotonically to the exact value as $dt \to 0$:

dt	MFPT ($\alpha = 0.5$, $\sigma_0 = 0.8$)	MFPT ($\alpha = 2.0$, $\sigma_0 = 0.5$)
0.02	12.3115 (+9.7%)	561.55 (+26.6%)
0.005	11.7708 (+4.9%)	503.71 (+13.5%)
0.001	11.3334 (+1.0%)	461.83 (+4.1%)
0.0005	11.2453 (+0.2%)	—

The Euler scheme with $\max(R, 0)$ truncation overestimates the MFPT because it delays arrival at the threshold. The bias is monotone in dt and consistent across regimes.

The second method is the censoring test [`verification/stochastics/censoring_test.py`]. In the first regime the process has no censoring within $T_{\max} = 300$: all 20,000 simulated paths reach R_0. In the second regime only 39.7% reach R_0 at $T_{\max} = 300$, and the tail of τ is geometric (ratio $P_{\text{emp}}/P_{\text{geo}} \approx 0.9927$ with std 0.0038 across five windows). The geometric tail implies that the unconditional MFPT is $T_0/(1-p)$ where $p = P(\tau > T_0)$, giving 635.7 in that regime.

The two methods agree on the qualitative structure: the first-passage time is finite, grows with R_0, and is dominated by the ratio α/σ_0^2. No claim of superexponential growth is made.

2.4 Stationary Distribution

Under the Feller condition $2\alpha\bar{R} \geq \sigma_0^2$, the CIR process has a unique stationary distribution:

$$R_\infty \sim \text{Gamma}(k, \theta), \quad k = \frac{2\alpha\bar{R}}{\sigma_0^2}, \quad \theta = \frac{\sigma_0^2}{2\alpha} \tag{2.6}$$

with moments

$$\mathbb{E}[R_\infty] = k\theta = \bar{R}, \quad \text{Var}[R_\infty] = k\theta^2 = \frac{\sigma_0^2\bar{R}}{2\alpha} \tag{2.7}$$

We verify (2.6) and (2.7) by simulating the process from $R(0) = \bar{R}$ over $T = 50$, discarding the first 50% of samples as burn-in, and comparing empirical moments against the theoretical values [`verification/stochastics/stationary_stats.py`]. The results:

Parameters	$\mathbb{E}[R_\infty]$ theory	empirical	Var theory	Var empirical
$\alpha = 0.5$, $\sigma_0 = 0.8$	1.000000	0.998296 (0.17%)	0.640000	0.637595 (0.38%)
$\alpha = 2.0$, $\sigma_0 = 0.5$	1.000000	1.000062 (0.006%)	0.062500	0.063097 (0.96%)

The relative differences are below 1% in all four quantities. The Z-score of the whitewashing target $R_0 = 2.2$ is 1.50σ in the first regime and 4.80σ in the second, consistent with empirical estimates (1.5050σ and 4.7770σ).

A two-sample Kolmogorov–Smirnov test against Gamma(k, θ) does not reject the stationary distribution at 95%: $D = 0.00354$, $p = 0.557$ (first regime); $D = 0.00479$, $p = 0.201$ (second regime). Because the simulated values are autocorrelated (50,000 paths × 2,500 steps), the KS p-value is indicative only. The moment comparison is the primary evidence.

2.5 Integro-Differential Formulation

For the Fokker–Planck density $p(r, t)$ of the reputation process, the forward equation is:

$$\frac{\partial p}{\partial t} = -\frac{\partial}{\partial r}[\mu(r)p] + \frac{1}{2}\frac{\partial^2}{\partial r^2}[\sigma^2(r)p] + \lambda \int_0^1 \left[\frac{p(r/y, t)}{y} - p(r, t)\right] \nu(dy) \tag{2.8}$$

with $\mu(r) = \alpha(\bar{R} - r)$, $\sigma^2(r) = \sigma_0^2 r$, and the jump term written for the multiplicative specification. The domain is $r \in (0, \infty)$, with absorbing boundary at $r = 0$.

The associated value function $C_R(r)$ for reputational collateral satisfies the integro-differential equation:

$$\frac{\partial C_R}{\partial t} + \mu(r)\frac{\partial C_R}{\partial r} + \frac{1}{2}\sigma^2(r)\frac{\partial^2 C_R}{\partial r^2} + \lambda \int_0^1 [C_R(ry) - C_R(r)]\nu(dy) - rC_R = -\pi(r) \tag{2.9}$$

with boundary condition $C_R(0) = 0$, interpreted in the viscosity sense. Existence and uniqueness follow from standard arguments for integro-PDEs with polynomial growth of π [10].

2.6 Summary of the Section

Claim	Status	Script
Additive jumps violate positivity	Verified	`sim_positivity.py`
Multiplicative jumps preserve positivity	Verified	`sim_positivity.py`
Exact MFPT via numerical integration	Verified	`mfpt_bound_explicit.py`
$C^* = 1$ is not a valid bound	Verified	`mfpt_bound_explicit.py`
Euler bias decreases with dt	Verified	`dt_convergence.py`
D4 tail is geometric; S2.2 has no censoring	Verified	`censoring_test.py`
Stationary distribution is Gamma(k, θ)	Verified	`stationary_stats.py`

The Section's quantitative content is limited to: positivity verification (40.58% vs 0.0000%); exact MFPT by numerical integration (11.2203 and 443.7374); implicit C^* per regime (0.562 and 3.14×10^{-8}); convergent simulated MFPT with dt (11.2453 at $dt = 5 \times 10^{-4}$); stationary moments (relative difference below 1%).

3 Tensor Compression

3.1 Tensor Representation of Reputation

Reputation is represented as a three-dimensional tensor

$$T \in [0, 1]^{C \times D \times H} \tag{3.1}$$

where C indexes chains ($C = 20$ in the reference configuration), D indexes contribution domains ($D = 5$), and H indexes time horizons ($H = 5$). Entry $T[c, d, h]$ is the validator's normalized score on chain c, in domain d, at horizon h.

The tensor has $C \cdot D \cdot H = 500$ entries in the reference configuration. This is the object whose norm is compared against the reputational threshold Θ_r in the zero-knowledge circuit of Section 4.

3.2 Tucker Decomposition

The Tucker decomposition of T [14] is

$$T \approx G \times_0 A^{(0)} \times_1 A^{(1)} \times_2 A^{(2)} \tag{3.2}$$

where $G \in \mathbb{R}^{R_1 \times R_2 \times R_3}$ is the core tensor and $A^{(0)} \in \mathbb{R}^{C \times R_1}$, $A^{(1)} \in \mathbb{R}^{D \times R_2}$, $A^{(2)} \in \mathbb{R}^{H \times R_3}$ are the factor matrices. We compute the decomposition via higher-order SVD (HOSVD), which takes the leading singular vectors of each mode unfolding.

For the reference configuration we use $(R_1, R_2, R_3) = (4, 3, 3)$. The storage required for the Tucker representation is

$$R_1 R_2 R_3 + R_1 C + R_2 D + R_3 H = 146 \text{ entries} \tag{3.3}$$

versus $C \cdot D \cdot H = 500$ entries for the uncompressed tensor. The storage ratio is 3.4247× (a 70.8% reduction) [`verification/tensor/tucker_analysis.py`].

3.3 Reconstruction Error

The HOSVD is exact for tensors of rank at most (R_1, R_2, R_3) and approximate otherwise. We quantify the reconstruction error

$$\varepsilon(T) = \frac{\|T - \hat{T}\|_F}{\|T\|_F} \tag{3.4}$$

for four tensor types at rank $(4, 3, 3)$:

Tensor type	$\|T\|_F$	$\|T - \hat{T}\|_F$	$\varepsilon(T)$
Clean low-rank	73.068	0.000	4.8×10^{-16}
Low-rank + noise ($\sigma = 0.05$)	45.261	0.993	2.19%
Random (zero mean)	6.407	5.493	85.74%
Blocky (community structure)	6.775	3.559	52.53%

Three observations follow.

First, the HOSVD implementation is exact: the clean low-rank tensor is reconstructed to machine precision (relative error 4.8×10^{-16}). This verifies the decomposition itself.

Second, the error for a low-rank tensor corrupted by noise is equal to the noise-to-signal ratio $\|N\|_F / \|T\|_F$. The decomposition captures the underlying low-rank component exactly and fails to capture only the noise, which is by construction not low-rank.

Third, and most importantly: **HOSVD does not capture block structure.** A tensor whose entries are constant within communities in mode C has mode-1 and mode-2 unfoldings of full rank. The truncation to $R_1 = 4$ rows discards most of the energy in those modes, and the reconstruction error is dominated by truncation rather than by noise. In the reference configuration the blocky tensor has error 52.53%, comparable to that of a random tensor.

The practical implication is that Tucker compression is efficient for tensors with low-rank structure and inefficient for tensors with community structure. Which regime the reputation tensor occupies is an empirical question that depends on the correlation of validator scores across chains, domains, and horizons. We do not resolve this question in this paper.

3.4 Error Versus Rank

For the low-rank noisy tensor, the reconstruction error decreases monotonically with rank:

(R_1, R_2, R_3)	Storage	Ratio	$\varepsilon(T)$
$(1, 1, 1)$	31	16.13×	5.02%
$(2, 2, 2)$	68	7.35×	3.24%
$(3, 3, 3)$	117	4.27×	2.32%
$(4, 3, 3)$	146	3.42×	2.19%
$(5, 5, 5)$	275	1.82×	1.52%

The trade-off is between storage reduction and reconstruction fidelity. The choice of rank is a design parameter; the reference configuration $(4, 3, 3)$ is a specific operating point, not an optimum.

3.5 Tucker Is Not in the Security Path

The Tucker compression is a storage and transmission optimization. It is not part of the trust chain of the protocol.

The zero-knowledge proof of Section 4 verifies a threshold on the norm of the reputation tensor. The proof operates on the committed tensor, not on its Tucker approximation. A prover who fabricates the factors G and $A^{(i)}$ cannot produce a sound proof of the committed tensor's norm, because the commitment binds the prover to the raw tensor.

Consequently the storage savings of 3.42× do not translate into a reduction of the ZK circuit cost. The circuit must verify the norm of the raw tensor, which is $O(C \cdot D \cdot H)$ constraints under the commitment scheme. Tucker is used only outside the circuit, to reduce the size of the data transmitted from the reputation oracle to the verifier.

This distinction corrects a claim in an earlier formulation of the protocol, in which the storage savings were presented as a reduction of the circuit cost. The two are independent: the storage reduction is real (3.4247×) and the circuit cost reduction is zero. The falsification log records this correction as F-012.

3.6 Summary of the Section

Claim	Status	Script
HOSVD is exact for low-rank tensors	Verified	`tucker_analysis.py`
Storage reduction $500 \to 146$	Verified	`tucker_analysis.py`
Error for noisy low-rank: $\|N\|/\|T\|$	Verified	`tucker_analysis.py`
Error for random: 85.7%	Verified	`tucker_analysis.py`
Error for blocky: 52.5% (HOSVD fails)	Verified	`tucker_analysis.py`
Tucker is not in the security path	Design	Sections 4, 5

The quantitative content of this Section is limited to the storage ratio and the reconstruction errors. The Tucker decomposition is a representation choice, not a security mechanism.

4 Zero-Knowledge Threshold Proof

4.1 Circuit Specification

The protocol requires validators to prove, without revealing the full reputation tensor T, that its Frobenius norm exceeds a threshold:

$$\|T\|_F \geq \Theta_r \tag{4.1}$$

The natural implementation is a zero-knowledge proof [6] over an R1CS circuit that verifies (4.1). The prover commits to T in some form and proves knowledge of a witness satisfying the inequality.

The original specification of the protocol used a Tucker-compressed witness: the prover supplies the core tensor G and factor matrices $A^{(0)}, A^{(1)}, A^{(2)}$, the circuit reconstructs $\hat{T} = G \times_0 A^{(0)} \times_1 A^{(1)} \times_2 A^{(2)}$, and verifies $\|\hat{T}\|_F \geq \Theta_r$. This specification is unsound.

4.2 Soundness Failure

The circuit accepts a proof of knowledge of some tensor $\hat{T}$ whose norm exceeds Θ_r. It does not check that $\hat{T}$ equals any committed tensor. A prover with zero reputation can therefore choose G and the factor matrices at will and produce a valid proof for any threshold.

We demonstrate the exploit by constructing a tensor with true norm below the threshold and a fabricated tensor with norm above it [`verification/zk/soundness_zk_check.py`]:

Tensor	$\|\cdot\|_F$	Threshold	Accepted
True reputation	0.223607	0.5	No
Fabricated (prover-chosen)	1.000000	0.5	Yes

The exploit is trivial. Groth16 [7] is sound with respect to the circuit; the circuit verifies a property of a value the prover controls completely. There is no commitment binding the prover to any specific tensor.

4.3 Repair Options

Four designs are considered, three of which are sound [`verification/zk/soundness_zk_check.py`]. We add a fifth column with the on-chain gas cost of verification, measured locally with Foundry [`integrations/avalanche/benchmarks/src/GasBenchmarks.sol`]. The primitive costs on the EVM are

Primitive	Gas
`ecAdd` (0x06)	292
`ecMul` (0x07)	6,142
`ecPairing` (0x08)	$45{,}000 + 34{,}000k + 142$

with k the number of pairings. Groth16 verification uses $k = 4$, giving 181,142 gas for the pairing step alone. These measurements match EIP-196 and EIP-197 exactly.

Design	Sound	Constraints	Proof size	On-chain gas (direct)
As specified (Tucker, no commitment)	No	573	0.20 KB	∼206k
Option A: Merkle commitment, full opening	Yes	1,080,501	0.20 KB	∼470M
Option A′: Poseidon commitment over full tensor	Yes	240,261	0.20 KB	∼30M
Option B: Pedersen + Bulletproofs	Yes	0	0.72 KB	∼6.5M

The column "On-chain gas (direct)" measures the cost of performing the verification work on-chain without a recursive SNARK. For the first row it is the cost of verifying the Groth16 proof of the small circuit (∼206k, constant). For options A, A′, and B it is the cost of the on-chain work the SNARK would otherwise do: hashing the tensor (A, A′) or performing the Bulletproofs scalar multiplications (B). With recursion, all three options collapse to ∼206k on-chain; the column shows why recursion is not optional.

The constraint count for the unsound design decomposes as

$$R_1R_2R_3 + R_1R_2R_3 + CDH + 1 = 36 + 36 + 500 + 1 = 573 \tag{4.2}$$

covering the Tucker factor products, the reconstruction of $\hat{T}$, the sum of squares, and the final comparison. This is an order-of-magnitude estimate of the arithmetic complexity; it is not an exact Groth16 constraint count.

Option A commits to T with a Merkle tree of Poseidon hashes and opens all $N = C \cdot D \cdot H$ leaves inside the circuit. The circuit requires 1,080,501 constraints. Direct on-chain verification of the resulting Groth16 proof costs approximately 206k gas if the circuit fits in one Groth16 proof, which it does not: 1.08M constraints produce a proving key and a proving time that are not practical for on-chain verification [CONJECTURE: the exact key size and proving time depend on the Groth16 implementation and are not measured in this paper]. On-chain direct verification is not viable.

Option A′ commits to T with a Poseidon tree over the entire tensor inside the circuit. With 240,261 constraints the circuit is still too large for practical Groth16 proving on consumer hardware.

Option B replaces the R1CS circuit with a Pedersen commitment to the flattened tensor and a Bulletproofs inner-product argument. The proof size for $n = 500$ elements is $O(\log n) \approx 0.72$ KB. The R1CS constraint count is zero because there is no circuit; soundness is provided by the special-soundness of the Bulletproofs protocol and the binding property of the Pedersen commitment under the discrete logarithm assumption. Direct verification on-chain requires $2N$ scalar multiplications and $2N$ point additions, or approximately 6.5M gas for $N = 500$. This is 31× the cost of Groth16 verification.

None of the three sound designs admits direct on-chain verification at acceptable cost. Option A and Option A′ are prohibitively large at the constraint level. Option B is prohibitively expensive at the gas level. The viable path is recursive SNARK composition, treated in Section 4.4.

4.4 Recursive Verification

The Bulletproofs verification, which is $O(N)$ in scalar multiplications and costs ∼6.5M gas for $N = 500$ if performed on-chain, is instead performed off-chain. A recursive SNARK [7] then attests that the off-chain verification was correct, and only the recursive proof is posted on-chain.

The recursive proof has constant size and constant verification cost. For a Groth16 recursion over BN254, the on-chain cost is approximately 250k gas, dominated by the `ecPairing` step at 181,142 gas. This is the same order of magnitude as the cost of verifying a single Groth16 proof directly.

We do not measure the proving time of the recursive proof in this paper. The recursive circuit is larger than the inner verification circuit by a factor that depends on the SNARK used for recursion. For the reference configuration, we estimate the recursive proving time in the range of tens of seconds on a modern CPU, but the estimate is not verified.

The choice of recursion framework (Groth16 recursion, Plonk recursion, Halo2 recursion) is not fixed in this paper. All three produce constant-size on-chain proofs at similar gas cost; the difference is the proving time and the trusted setup requirements.

The protocol adopts Option B with recursive SNARK composition. Selecting it does not remove the Tucker representation from the protocol: Tucker remains useful for data transmission outside the circuit (Section 3), but the circuit operates on the raw committed tensor.

4.5 Estimates and Their Provenance

The gas measurements for the EVM precompiles are exact: they match EIP-196 and EIP-197 to the gas unit. The Poseidon-2 measurement of 104,963 gas is an upper bound from an unoptimized Solidity implementation; production implementations are 20–40k gas. The relevant constraint on Poseidon in this paper is the R1CS count of 240 per hash, which is independent of the on-chain gas cost.

The recursive SNARK cost of ∼250k gas is an estimate. The Bulletproofs proof size of 0.72 KB for $n = 500$ is an estimate from the literature. The proving time of the recursive proof is not measured.

The exploit demonstration of Section 4 is not an estimate. The values 0.223607 and 1.000000 are computed by the script.

4.6 Summary of the Section

Claim	Status	Script
Original circuit is unsound	Verified	`soundness_zk_check.py`
Fabricated tensor passes threshold	Verified	`soundness_zk_check.py`
Constraint count for unsound design	Arithmetic	`soundness_zk_check.py`
Merkle opening: $\sim 1.08 \times 10^6$ constraints	Arithmetic	`soundness_zk_check.py`
Poseidon tree: $\sim 2.4 \times 10^5$ constraints	Arithmetic	`soundness_zk_check.py`
Bulletproofs: 0 R1CS, ∼0.72 KB	Estimate	`soundness_zk_check.py`
Groth16 $k = 4$ pairing: 181,142 gas	Verified	`GasBenchmarks.sol`
Bulletproofs direct on-chain: ∼6.5M gas	Estimated	`GasBenchmarks.sol`
Recursion required for on-chain viability	Design	—

The Section demonstrates the unsoundness of the original design with a concrete exploit, and recommends a replacement. The exact cost of the replacement depends on implementation parameters not fixed in this paper; the arithmetic is correct given those parameters.

5 Collusion Detection

5.1 The Collaboration Graph

Validators interact through shared governance votes, co-authored pull requests, and simultaneous liquidity events [8, 16]. We model these interactions as a symmetric weighted graph with adjacency matrix

$$W \in \mathbb{R}^{n \times n} \tag{5.1}$$

where W_{ij} counts the interactions between validators i and j. The graph has $n = 500$ nodes in the reference configuration, with mean degree $\bar{d} = 20$.

The hypothesis of interest is that a subset of validators coordinates to inflate each other's reputation without genuine contribution. In graph terms, this appears as a dense subgraph (a clique) embedded in a sparser background.

5.2 The Normalization Problem

A natural spectral approach analyzes the symmetric normalized adjacency matrix

$$A_{\text{sym}} = D^{-1/2} W D^{-1/2} \tag{5.2}$$

where $D = \text{diag}(d_1, \ldots, d_n)$, $d_i = \sum_j W_{ij}$. This matrix has a bulk spectrum that recovers a Wigner-type distribution [15, 1] for heavy-tailed graphs, which the original formulation of the protocol used for detection.

The normalization is inappropriate for detecting cliques. Each edge is scaled by $1/\sqrt{d_i d_j}$. Because a clique raises the degree of its members, the normalization downweights the clique's internal edges relative to the background. Quantitatively, for a clique of size $m = 20$ with intra-clique density $10\times$ the background:

Quantity	Value
Degree of a clique node	≈ 27.2
Degree of a background node	≈ 20.0
Normalized intra-clique edge	$1/27.2 \approx 0.037$
Normalized background edge	$1/20.0 \approx 0.050$

The clique's edges are weaker after normalization than the background's. The density signal is cancelled by the degree increase.

We verify this numerically [`verification/rmt/clique_spectra.py`]. For a single clique of size $m = 20$ with intra-clique density $10\times$ the background, the three candidate matrices give the following leading non-trivial eigenvalue:

Matrix	Statistic	λ (H_0)	λ (H_1)	Bulk edge	Gap (H_1)
W (raw)	$s[-2]$	8.6896	10.6439	8.9443	+1.6996
A_{sym}	$s[-2]$	0.4254	0.4316	0.4472	−0.0156
B	$s[-1]$	8.6896	10.7011	8.9443	+1.7569

The choice of statistic differs by matrix. For W and A_{sym}, $s[-1]$ is the Perron eigenvalue (trivial, equal to $d_{\max}$ and 1 respectively) and $s[-2]$ is the leading non-trivial direction. For B, the identity $B \cdot \mathbf{1} = 0$ places the trivial eigenvalue at the center of the spectrum, and $s[-1]$ is the signal. The change is documented in falsification log entry F-018.

The table shows that W separates the clique from the bulk (gap +1.6996 against the asymptotic edge $2\sqrt{\bar{d}} = 8.9443$). A_{sym} does not (gap −0.0156 against its bulk edge 0.4472). B separates the clique by a wider gap than W (+1.7569 vs +1.6996) and exceeds both the theoretical bulk edge and the empirically calibrated threshold $q_{99} = 9.0542$ of Section 5.4.

The relationship between B and W is explained by a structural identity. For a regular graph with degree d, the modularity matrix is $B = A - (d/n) \cdot \mathbf{1}\mathbf{1}^\top$, and since $\mathbf{1}$ is orthogonal to every other eigenvector, the spectrum of B equals that of A with the Perron eigenvalue replaced by 0. Consequently $\lambda_{\max}(B) = \lambda_2(W)$ exactly. The Erdős–Rényi graph used in the null simulation is almost regular (degrees 10–33, mean 19.88, std 4.22), so the identity holds to within 10^{-5} in H_0. In H_1 the clique raises the degrees of its 20 members, breaking the regularity and leaving B slightly ahead of W.

This is the content of the falsification log entry F-002.

5.3 The Modularity Matrix

The matrix that resolves both problems is the modularity matrix

$$B = W - \frac{dd^\top}{2m} \tag{5.3}$$

where d is the degree vector and $m = \frac{1}{2}\sum_i d_i$ is the number of edges. The matrix B satisfies

$$B \cdot \mathbf{1} = 0 \tag{5.4}$$

which removes the degree bias by construction. Its bulk spectrum under the null is centered at zero with edge $2\sqrt{\bar{d}}$ for the degree-corrected stochastic block model.

The modularity matrix is the standard tool for community detection in the stochastic block model [3, 4]. It has the property that the leading eigenvector outside the bulk corresponds to the community structure, not to the degree distribution.

5.4 Null Distribution and Calibration

We calibrate the null distribution of $\lambda_{\max}(B)$ and $\lambda_{\min}(B)$ by Monte Carlo on Erdős–Rényi graphs with $n = 500$ and $\bar{d} = 20$ [`verification/rmt/calibration_modularity.py`]. The calibration uses $N_{H_0} = 2000$ replicates with seed 42.

Statistic	Mean	Std	q_{99}
$\lambda_{\max}(B)$ (right tail, cliques)	8.7679	0.1115	9.0542
$-\lambda_{\min}(B)$ (left tail, bipartite)	8.8601	0.1101	9.1371

The empirical thresholds at significance level α are:

α	Reject H_0 if
0.01	$\lambda_{\max}(B) > 9.0542$ or $-\lambda_{\min}(B) > 9.1371$
0.05	$\lambda_{\max}(B) > 8.9663$ or $-\lambda_{\min}(B) > 9.0428$

The dual-tail structure distinguishes clique-based collusion (right tail) from bipartite polarization (left tail). The right-tail threshold is calibrated at $q_{99} = 9.0542$, which is 2.57σ above the null mean. The left-tail threshold is calibrated at 9.1371, which is 2.52σ above its null mean.

The statistic for the right tail is $s[-1]$, the largest eigenvalue of B, not $s[-2]$. The change is documented in falsification log entry F-018: for a balanced two-block structure the signal is at $s[-1] = 17.5$ while $s[-2] = 8.5$ sits inside the bulk. The "skip Perron" convention that motivates $s[-2]$ in the adjacency matrix does not apply to the modularity matrix, where $B \cdot \mathbf{1} = 0$ places the trivial eigenvalue at the center of the spectrum.

5.5 Detection Power

We measure the power of the right-tail test against clique-based collusion by simulating graphs with k planted cliques of size m and intra-clique density $10\times$ the background [`verification/rmt/calibration_modularity.py`]. The measurement uses $N_{H_1} = 500$ replicates per configuration with Wilson 95% confidence intervals.

Configuration	$\lambda_{\max}(B)$ mean	TPR @ $\alpha = 0.01$	95% CI
$k = 1$, $m = 20$	9.7452	0.972	$[0.954, 0.983]$
$k = 3$, $m = 20$	10.2714	1.000	$[0.992, 1.000]$
$k = 5$, $m = 20$	10.5294	1.000	$[0.992, 1.000]$
$k = 1$, $m = 40$	14.2441	1.000	$[0.992, 1.000]$
$k = 3$, $m = 40$	16.2281	1.000	$[0.992, 1.000]$

Detection is uniform across the five configurations. The lowest true positive rate is 0.972, for a single clique of size 20 with intra-clique density $10\times$ the background. All other configurations are detected at 1.000.

The abrupt-transition pattern that appeared with $s[-2]$ was an artifact of the wrong statistic. With $s[-1]$, the smallest clique is detected at 97.2% and every other configuration at 100%. There is no intermediate regime in the simulated range.

The single-clique case at $m = 20$ remains the tightest: 0.972 with a Wilson 95% confidence interval of $[0.954, 0.983]$. The interval is above 0.9, so the detection is reliable. Configurations below $m = 20$ were not simulated; the boundary at which detection degrades is below the tested range.

Comparison with A_{sym}. The same test run on the normalized adjacency matrix A_{sym}, with the same clique configuration ($k = 1$, $m = 20$ on $n = 500$, $\bar{d} = 20$), gives a true positive rate of 0.033 at $\alpha = 0.01$ [`verification/rmt/archive/calibracion_rmt.py`], measured over $N_{H_1} = 30$ replicates. The modularity matrix B gives 0.972 on the same configuration, measured over $N_{H_1} = 500$ replicates. The ratio is approximately $29.2\times$. This is the numerical basis for the design decision to replace A_{sym} with B, documented as falsification log entry F-002.

The two Wilson 95% confidence intervals do not overlap: A_{sym} is $[0.006, 0.167]$ and B is $[0.954, 0.983]$ [`verification/rmt/wilson_intervals.py`]. The comparison is statistically significant at 95% despite the small sample size of the A_{sym} measurement ($N = 30$).

5.6 The Detection Boundary

The detection power measured in Section 5.4 is uniform across the simulated range. All five configurations have true positive rates above 0.97. There is no observable transition from partial to full detection within the tested parameter range.

This is a consequence of using $s[-1]$ as the statistic. With the previous statistic $s[-2]$, the pattern appeared to be controlled by the ratio $k \cdot m^2/n$ and a boundary around $k \cdot m^2/n \approx 1.5$ was conjectured. That pattern was an artifact: with $s[-1]$ the same configurations all sit above the (much lower) effective boundary.

[CONJECTURE] The detection boundary for the modularity-spectrum test with $s[-1]$ on the degree-corrected stochastic block model lies below the configurations tested here. We do not locate it precisely; the smallest configuration tested (a single clique of size $m = 20$) is detected at 0.972, and configurations smaller than $m = 20$ were not simulated. A derivation of the boundary from the Kesten–Stigum threshold or from the random matrix literature would be a separate contribution.

Consequently, collusion at the scale tested here is detectable from the collaboration graph alone. Collusion below the (unlocated) boundary would not be. The protocol relies on economic deterrence through slashing (Section 7) and reputational collateral destruction (Section 2) for the sub-boundary regime, which is not characterized in this paper.

5.7 Limitations

The calibration is performed on Erdős–Rényi and Chung–Lu graphs. Real collaboration graphs have additional structure not captured by these models: community hierarchies, temporal correlations between interactions, and directionality (who initiates contact). The empirical quantiles in Section 5.4 are valid for the calibration distribution; their validity on real graphs is an open question.

The left tail ($-\lambda_{\min}$) is measured on disassortative block structures [`verification/rmt/bipartite_power.py`]. The true positive rate is 0.990–1.000 across block configurations from balanced (250/250) to heavily unbalanced (50/450), with density ratios $\rho_{\text{between}} \in \{2, 5, 10\}$ and $N_{H_1} = 500$ replicates. The left-tail test has power comparable to the right-tail test.

An adversarial validator who knows the test can attempt to avoid detection by keeping the clique size below the (unlocated) boundary, mixing clique members with honest interactions, or avoiding clique structures entirely. The test has power against the naive adversary (uniform cliques and bipartite structures of moderate size); its power against an adaptive adversary below the boundary is not characterized.

5.8 Summary of the Section

Claim	Status	Script
A_{sym} masks cliques	Verified	`clique_spectra.py`, `sim_laplacian.py`
Modularity B with $s[-1]$ detects cliques	Verified	`calibration_modularity.py`
Null q_{99} (right tail, $s[-1]$) = 9.0542	Verified	`calibration_modularity.py`
TPR $k = 1$, $m = 20 = 0.972$	Verified	`calibration_modularity.py`
TPR $k = 3$, $m = 20 = 1.000$	Verified	`calibration_modularity.py`
TPR $k = 1$, $m = 40 = 1.000$	Verified	`calibration_modularity.py`
A_{sym} vs B ratio = 29.2×	Verified	`wilson_intervals.py`
Detection boundary below tested range	Conjecture	—
Left-tail power measured	Verified	`bipartite_power.py`

The Section's quantitative content is limited to the calibration of the null distribution, the power measurements for both tails, and the demonstration that A_{sym} fails where B succeeds.

6 Payment Rule

6.1 The Revelation Problem

A validator is admitted to the validator set if its hybrid collateral exceeds the threshold:

$$\mathrm{CoA}(i) = S_f(i) + C_R(T(i)) \geq \Theta(t) \tag{6.1}$$

where S_f is the financial stake, C_R is the reputational collateral, and T is the reputation tensor.

The allocation of collateral between its financial and reputational components depends only on T. It does not depend on any private information that the validator might hold. This simplifies the mechanism design problem relative to a classic auction, but it also means that the standard tools of auction theory do not apply directly.

We address the two candidate sources of private information below.

6.2 Verified Reputation

The reputation tensor T is verifiable on-chain via the zero-knowledge threshold proof of Section 4. A validator whose reputation exceeds the threshold can prove it without revealing the tensor itself. A validator whose reputation falls below the threshold cannot produce a valid proof.

Consequently, T is not a private type in the sense of mechanism design. The validator cannot misreport it: the proof binds the validator to a specific tensor, and the verifier checks the binding. The revelation principle does not apply because there is nothing to reveal.

6.3 Private Cost of Capital

The validator's cost of capital $\omega \in [\underline{\omega}, \bar{\omega}]$ is private. It represents the opportunity cost of locking financial stake. A validator with low ω can lock stake cheaply; a validator with high ω faces a higher opportunity cost.

The allocation rule is

$$S_f^*(T) = \max\{\mathrm{VaR}_{\mathrm{tech}}, \Theta(t) - C_R(T)\} \tag{6.2}$$

which does not depend on ω. The type affects the validator's utility but not its allocation.

The utility of a validator of type ω with reputation T is

$$U(\omega, T) = \pi_{\mathrm{val}} - \omega \cdot S_f^*(T) - F_0 \tag{6.3}$$

where π_{val} is the expected validation revenue per epoch and F_0 is the fee charged for participation.

6.4 The Uniform Posted Price

Since the allocation does not depend on ω, the only mechanism design decision is the participation fee F_0. We choose a uniform posted price: all validators pay the same F_0, regardless of type.

The optimal F_0 is the largest value that preserves Individual Rationality for all types. The worst-case type is $(\bar{\omega}, T = 0)$, which combines the highest cost of capital with the lowest reputation:

$$U(\bar{\omega}, 0) = \pi_{\mathrm{val}} - \bar{\omega} \cdot \Theta - F_0 \geq 0 \tag{6.4}$$

Solving for F_0 at equality:

$$F_0^* = \pi_{\mathrm{val}} - \bar{\omega} \cdot \Theta \tag{6.5}$$

With the reference parameters ($\pi_{\mathrm{val}} = 0.15$, $\bar{\omega} = 0.10$, $\Theta = 1.0$):

$$F_0^* = 0.15 - 0.10 \cdot 1.0 = 0.0500 \tag{6.6}$$

The fee is charged per epoch and is the same for all participating validators.

6.5 Individual Rationality

The posted price F_0^* saturates IR at the worst-case type by construction. We verify numerically that IR holds for all types in the support and that the saturation is exact [`verification/mechanism/fee_specification.py`].

The Monte Carlo draws $N_{\text{VALIDATORS}} = 1000$ validators with $\omega \sim U[0.01, 0.10]$ and T uniform on $[0, 3]$ (so $C_R(T) \in [0, 0.6]$). The measured utilities are:

Statistic	Value
$U_{\min}$	0.009747
U_{mean}	0.069152
$U_{\max}$	0.095891
Fraction with $U \geq 0$	1.0000

The analytical saturation $U(\bar{\omega}, 0) = 0$ is exact, with residual below 10^{-9}. The Monte Carlo $U_{\min}$ is strictly positive (0.009747) because the specific pair $(\omega = 0.10, T = 0)$ is not sampled exactly in a finite draw. The bound holds analytically for the worst-case type and empirically for the sampled population.

6.6 Sensitivity

If the fee exceeds F_0^*, IR is violated for the worst-case types. We verify this numerically:

F_0	Fraction with $U < 0$	Revenue per epoch
0.025	0.0000	25.00
0.040	0.0000	40.00
0.050	0.0000	50.00
0.060	0.0020	60.00

At $F_0 = 0.06$ (20% above F_0^*), 0.2% of the sampled validators violate IR. The revenue trade-off is linear: increasing the fee above F_0^* gains revenue at the cost of excluding the worst-case types.

6.7 Incentive Compatibility

Incentive Compatibility is tautological in this mechanism. There is no private type to misreport: T is verified on-chain, and $S_f^*(T)$ does not depend on ω. A validator cannot improve its allocation by misreporting, because the allocation is determined by a public, verifiable signal.

The falsification log records this as F-006. The original formulation of the protocol applied Myerson's optimal auction rule [9], which requires a private one-dimensional type. The current mechanism does not have one. The payment rule is a saturated uniform posted price, not a Myerson auction.

6.8 Summary of the Section

Claim	Status	Script
T is verified, not private	Design	Section 4
$S_f^*(T)$ independent of ω	Design	—
$F_0^* = \pi_{\text{val}} - \bar{\omega}\Theta$	Analytical	`fee_specification.py`
IR holds for all types	Verified	`fee_specification.py`
IR saturation exact at worst-case type	Verified	`fee_specification.py`
$F_0 = 1.2F_0^*$ violates IR for 0.2%	Verified	`fee_specification.py`
IC is tautological	Logical	—

The payment rule is a uniform posted price, not a Myerson auction. The absence of a private type with respect to the allocation rule makes the mechanism simpler than standard auction theory would suggest.

7 Data Layer

7.1 Ingestion of Consensus Events

The reputation tensor T introduced in Section 3 is not declared by the validator. It is derived from consensus events on the source chains [12]. For each chain c, domain d, and horizon h, the entry $T[c, d, h]$ is computed from a formula frozen at genesis:

$$T[c, d, h] = f_c(\text{events in window } h) \tag{7.1}$$

where f_c is a public function that maps the observed consensus events to a normalized score in $[0, 1]$. The formula is not changeable by the validator; it is fixed before deployment and applies identically to all validators on chain c.

The events themselves are consensus outputs: signed attestations, governance votes, and liquidity events that have been finalized on the source chain. They are not subjective judgments or off-chain declarations.

7.2 Transport and Freshness

Entries of T are committed to the destination chain via state inclusion proofs against signed headers of the source chains. This requires a light client or a zero-knowledge bridge for each source chain. The protocol does not define the transport mechanism; it requires that the destination contract can verify inclusion proofs against signed source-chain state.

The tensor T is frozen per epoch. Two consecutive epochs cannot differ in more than $\gamma \cdot N$ entries, where $N = CDH$ and γ is a freshness parameter. The freshness constraint bounds the propagation of undetected fraud across epochs.

7.3 The Fraud Proof Mechanism

Any party can submit a fraud proof consisting of two state inclusion proofs against the same source-chain root that yield contradictory values for a single entry $T[c, d, h]$. If the proof is valid, the entry is marked fraudulent and the honest reporter receives a bounty B. The validator whose entry was fraudulent is slashed.

We model the audit equilibrium as follows. Let N_{ATT} be the number of attestations per epoch, p_{fraud} the fraction that are fraudulent, and c_{audit} the cost of auditing one attestation. Each verifier decides whether to audit based on the expected revenue [11]:

$$\mathbb{E}[\text{revenue}] = \frac{p_{\text{fraud}} \cdot N_{\text{ATT}} \cdot B}{n_{\text{auditors}}} - c_{\text{audit}} \tag{7.2}$$

where n_{auditors} is the number of active auditors. A verifier enters if $\mathbb{E}[\text{revenue}] > 0$ and exits otherwise. The equilibrium is

$$n^*(B) = \left\lfloor \frac{B}{B^*} \right\rfloor \tag{7.3}$$

with the threshold fee

$$B^* = \frac{c_{\text{audit}}}{p_{\text{fraud}}} \tag{7.4}$$

For $B < B^*$ no auditor enters; for $B \geq B^*$ at least one auditor is active. The threshold B^* is the fee at which a single auditor is indifferent between auditing and not.

7.4 Numerical Verification

We verify (7.3) and (7.4) by Monte Carlo [`verification/data-layer/simulate_data_layer_v3.py`]. The reference parameters are $N_{\text{ATT}} = 500$, $c_{\text{audit}} = 0.10$, $p_{\text{fraud}} = 0.10$, $V_{\text{fraud}} = 1.0$, $S_{\text{slash}} = 10.0$, $N_{\text{VERIFIERS}} = 20$. The threshold is $B^* = 0.10/0.10 = 1.0$.

B	$n^*(B)$	Detection	Attacker profit	Verifier profit	Integrity
0.75	0	0.000	+49.91	0.00	0.9002
1.00	1	1.000	−502.00	+0.20	1.0000
2.00	2	1.000	−502.00	+0.20	1.0000
5.00	5	1.000	−502.00	+0.20	1.0000
10.00	10	1.000	−502.00	+0.20	1.0000
20.00	20	1.000	−502.00	+0.20	1.0000

The table reports the interior equilibrium points, where B is either below the threshold or an exact integer multiple of B^*. The transition at $B = B^*$ is exact. For B below the threshold no verifier audits; the attacker earns $V_{\text{fraud}} \cdot N_{\text{ATT}} \cdot p_{\text{fraud}}$ per epoch without friction. For B at or above the threshold the auditor coverage reaches 100% in this model, and the attacker's expected profit is negative because the slashing dominates the fraud gain.

At the interior equilibrium points the analytical verifier profit is zero: the per-auditor revenue $D \cdot B/n^*$ equals the per-auditor cost $N_{\text{ATT}} \cdot c_{\text{audit}} = 50$ exactly when $n^* = B/B^*$. The observed +0.20 is the Monte Carlo deviation from zero, equal to $(D_{\text{obs}} - D_{\text{theory}}) \cdot B/n^*$. With $D_{\text{obs}} \approx 50.2$ and $D_{\text{theory}} = 50.0$, the deviation is $0.2 \cdot B^* = 0.20$. Between interior points the verifier profit is strictly positive; the value depends on B and is tabulated in the script output.

7.5 Slashing and Integrity

The attacker profit is negative for all $B \geq B^*$ because the slashing parameter $S_{\text{slash}} = 10.0$ exceeds the fraud gain $V_{\text{fraud}} = 1.0$ by an order of magnitude. The design of the slashing parameter is outside the scope of this section; the relevant observation is that once the audit threshold is crossed, the fraud is deterred.

The reputation of a slashed validator is reduced according to the process of Section 2. The combination of financial slashing and reputation destruction makes fraud strictly unprofitable for any B above the threshold.

7.6 Freshness as Second Line of Defense

The freshness parameter γ bounds the propagation of undetected fraud across epochs. If a fraudulent entry is present in epoch t, it can propagate to epoch $t+1$ only within the γ fraction allowed. Over k epochs, the maximum fraction of fraudulent entries in the tensor is bounded by

$$\text{fraud fraction} \leq \gamma \cdot k \tag{7.5}$$

in the worst case, where the adversary can insert fresh fraud each epoch. The growth is linear, not exponential. For $\gamma = 0.10$ and $k = 10$ epochs, the maximum fraudulent fraction is 1.0; for $\gamma = 0.05$ the maximum is 0.5 after 10 epochs. The freshness constraint prevents a single undetected fraud from contaminating the entire tensor.

7.7 Summary of the Section

Claim	Status	Script
$B^* = c_{\text{audit}}/p_{\text{fraud}}$	Analytical	`simulate_data_layer_v3.py`
$n^*(B) = \lfloor B/B^* \rfloor$ equilibrium	Verified	`simulate_data_layer_v3.py`
Attacker profit -502 with $S_{\text{slash}} = 10$	Verified	`simulate_data_layer_v3.py`
Integrity $= 1.0$ when $n^* \geq 1$	Verified	`simulate_data_layer_v3.py`
Freshness bounds propagation linearly	Analytical	—

The Section describes the ingestion, transport, and fraud-proof mechanism of the data layer, and verifies the audit equilibrium by Monte Carlo. The freshness parameter provides a second line of defense in case the fraud proof mechanism fails to detect a particular attack.

8 Bootstrap Dynamics

8.1 The Genesis Problem

At the genesis block, no validator has reputation. The reputation tensor T is identically zero and the reputational collateral $C_R(0)$ is zero. The protocol therefore begins as a pure proof-of-stake system with full financial stake requirements, and transitions to a hybrid regime as validators accumulate reputation.

The bootstrap question is: does the transition concentrate reputation in the early validators, and if so, how much? A mechanism that concentrates reputation in a small set of genesis validators would recreate the centralization it was designed to avoid.

8.2 Reputation Models

The dynamics of reputation during the bootstrap determine the outcome. We compare three models in earlier work and select one for the paper. The models differ in how a validator's activity a translates into reputation growth:

Model	Gain rule	Stationary value
Saturating	$k \cdot a \cdot (1 - R)$	$R \to 1$ for all a
Decreasing	$k \cdot a \cdot (1 - R)^2$	$R \to 1$ for all a
net_gain	$g \cdot a \cdot (1 - R) - \mu R$	$R^* = ga/(ga + \mu)$

The saturating and decreasing models converge to $R \to 1$ for all validators regardless of activity. After a sufficiently long horizon, the reputation is identical for all validators and carries no information about contribution. This is the content of the earlier comparison [`verification/bootstrap/compare_reputation_models.py`].

The net_gain model has a stationary value that depends on activity:

$$R^* = \frac{g \cdot a}{g \cdot a + \mu} \tag{8.1}$$

The correlation between activity and reputation at stationarity is 0.76 in the reference simulation, versus 0.08–0.11 for the saturating models [`verification/bootstrap/compare_reputation_models.py`]. We therefore use net_gain throughout the bootstrap analysis.

The sweep over μ at fixed $g = 0.01$ [`verification/bootstrap/sweep_net_gain.py`] gives a maximum of R_{std} at $\mu/g = 1.5$. We use $g = 0.01$, $\mu = 0.015$.

8.3 Bootstrap Simulation

We simulate the bootstrap from $R(0) = 0$ over 500 epochs [`verification/bootstrap/simulate_bootstrap_net_gain.py`]. Parameters:

- $N_{\text{MAX}} = 100$ validators, $N_{\text{INIT}} = 10$ at genesis.
- Entry rate 0.03 per epoch for new validators.
- Activity $a \sim \text{LogNormal}(0, 0.5)$ per validator.
- Transition rate $\alpha(t) = \min(1, t/200)$, reaching 1 at epoch 200.

We compare three policies:

- **pure_pos**: $S_f = \Theta$ throughout. No reputation adjustment.
- **hybrid_no_decay**: $S_f(t) = \max\{\text{VaR}_{\text{tech}}, \Theta - \alpha(t) \cdot C_R(R(t))\}$.
- **hybrid_decay**: same as hybrid_no_decay with R decaying at half-life 50 epochs.

8.4 Results

Policy	$t(\alpha{=}1)$	Gini	Top-10	S_f med	R med	R max
pure_pos	500	0.1283	0.1367	1.0000	0.3936	0.5976
hybrid_no_decay	200	0.1283	0.1367	0.7764	0.3936	0.5976
hybrid_decay	200	0.1563	0.1477	0.8393	0.2553	0.4348

The S_f med column measures the median financial stake required at the end of the simulation. Under pure_pos it is $\Theta = 1.0$. Under hybrid_no_decay it is 0.7764, a reduction of 22.4%. Under hybrid_decay it is 0.8393, a reduction of 16.1%.

The R med column is the median reputation. Under net_gain it converges to the stationary value $R^* = ga/(ga + \mu)$. For a validator with median activity $a \approx 1$, $R^* \approx 0.4$, consistent with the measured 0.3936.

The Gini and Top-10 columns are identical between pure_pos and hybrid_no_decay because both policies track the same reputation trajectory $R(t)$. The difference between them is in the financial stake policy $S_f(t)$, not in the reputation. Gini and Top-10 are computed on the distribution of R, not on S_f.

8.5 Comparison of Policies

Two observations follow from the table.

Decay does not reduce concentration. The hybrid_decay policy has higher Gini (0.1563) than hybrid_no_decay (0.1283). This is the opposite of the intuition that decay limits accumulation. The reason is that decay lowers the ceiling ($R_{\max}$ drops from 0.5976 to 0.4348) but also lowers the floor, spreading the distribution. Under hybrid_no_decay, the top validators saturate near $R \approx 0.6$ and the distribution compresses; under hybrid_decay, they fall back to $R \approx 0.43$ and the distribution widens. The falsification log records this as F-015.

hybrid_decay is dominated on both metrics. It has higher Gini (0.1563 vs 0.1283) and higher median S_f (0.8393 vs 0.7764). The only advantage is lower $R_{\max}$, which could matter if the protocol has a constraint on maximum reputation. Absent such a constraint, hybrid_no_decay is the recommended policy.

8.6 Recommendation

The protocol uses hybrid_no_decay. The financial stake reduction under this policy is 22.4% at the median validator. This is lower than the 37% that an earlier analysis reported under the saturating model [see falsification log entry F-016], because the saturating model produces $R \to 1$ for all validators while net_gain produces $R^* \approx 0.4$.

The reduction is real but modest. It corresponds to a validator with median reputation needing to lock 77.6% of the stake it would need under pure proof-of-stake. Validators with above-median reputation lock less; validators below median lock more.

8.7 Summary of the Section

Claim	Status	Script
net_gain is the selected model	Verified	`compare_reputation_models.py`
$\mu/g = 1.5$ maximizes R_{std}	Verified	`sweep_net_gain.py`
S_f median 0.7764 under hybrid_no_decay	Verified	`simulate_bootstrap_net_gain.py`
22.4% stake reduction at median	Verified	`simulate_bootstrap_net_gain.py`
hybrid_decay raises Gini	Verified	`simulate_bootstrap_net_gain.py`
Hybrid_no_decay is recommended	Design	—

The Section's quantitative content is limited to the bootstrap simulation under the net_gain model and the comparison of three policies. The saturation of the alternative models is documented as F-016.

The selection of net_gain is based on its correlation with activity [5] (0.76 vs 0.08–0.11 for the saturating models), not on satisfying all four criteria of the model comparison. The standard deviation of reputation under net_gain is 0.0282 in `compare_reputation_models.py` and 0.0424 at the sweep optimum $\mu/g = 1.5$; both fall below the 0.05 threshold used in the comparison script. The threshold is a heuristic, not a requirement, and the net_gain model is chosen for the correlation property, which the saturating models fail by an order of magnitude.

9 Evaluation

9.1 Scope

This section evaluates three things: the reproducibility of the numerical claims, the measured on-chain gas costs of the cryptographic primitives, and the stability of the conclusions under parameter variation. Two benchmarks are deferred to future work because they require a live network: the proving time of the recursive SNARK, and the latency of cross-chain message propagation.

9.2 Reproducibility

Every quantitative claim in Sections 2 through 8 is accompanied by a script in `verification/`. The scripts are self-contained: they depend only on NumPy and SciPy, and they declare their seeds and sample sizes at the top of each file. Running `pip install -r verification/requirements.txt` followed by the commands listed in `verification/README.md` reproduces every number reported in the paper. The README lists the 22 scripts and their expected runtimes.

The project documents its own falsifications in `docs/audit/numerical-verification.md`. As of this writing the log contains 18 entries (F-001 through F-018), each one a claim that was made, tested, and corrected. The log exists because the numerical verification is the primary defense against errors of the kind that would otherwise propagate through the paper unchallenged.

The scripts run in the following time ranges on a 2020-era desktop:

Script	Time
`calibration_modularity.py`	∼400 s
`dt_convergence.py`	∼130 s
`sim_positivity.py`	∼34 s
`censoring_test.py`	∼17 s
`clique_spectra.py`	∼17 s
`stationary_stats.py`	∼12 s
All others	< 5 s each

Total runtime for the full suite: approximately 10 minutes on a single core. The dominant cost is the RMT calibration with $N_{H_0} = 2000$ and $N_{H_1} = 500$.

9.3 On-Chain Gas Costs

The cryptographic design of Section 4 has an on-chain component. We measure the gas cost of the EVM precompiles that implement the BN254 pairings and scalar multiplications [`integrations/avalanche/benchmarks/src/GasBenchmarks.sol`]. The measurements match EIP-196 and EIP-197 to the gas unit:

Primitive	Measured gas	EIP formula
`ecAdd` (0x06)	292	150 + overhead
`ecMul` (0x07)	6,142	6,000 + overhead
`ecPairing` $k = 1$	79,142	45,000 + 34,000+ overhead
`ecPairing` $k = 2$	113,142	45,000 + 68,000+ overhead
`ecPairing` $k = 4$	181,142	45,000 + 136,000+ overhead

The overhead is 142 gas in all five measurements, consistent with a single `staticcall` dispatch. Groth16 verification over BN254 uses $k = 4$ pairings and therefore costs 181,142 gas for the pairing step alone, plus approximately 25k gas for the surrounding verification logic. The total is approximately 206k gas, constant in the size of the circuit.

The direct verification of a Bulletproofs proof of Section 4 for $n = 500$ elements requires $2n$ scalar multiplications and $2n$ point additions: approximately 6.5M gas. This is 31× the cost of a Groth16 verification. The recursive SNARK of Section 4.4 reduces this to the constant cost of the recursive proof, approximately 250k gas.

The measurement uses a local Foundry instance with Solc 0.8.33. The precompile gas schedule is a constant of the EVM protocol, so the measurements apply identically to any EVM-compatible chain including Avalanche C-Chain.

9.4 Stability of Conclusions

The conclusions of the paper depend on parameter choices that are fixed at design time. We check that small variations do not change the qualitative outcome.

Reputation dynamics (Section 2). The domain validity of the multiplicative-jump specification is verified pathwise: 0 of 100,000 trajectories leave the positive domain for any parameter choice that satisfies Feller. The mean first-passage time converges to the exact value as $dt \to 0$, with a relative error of 0.22% at $dt = 5 \times 10^{-4}$. The conclusions do not depend on the discretization.

Collusion detection (Section 5). The null quantile $q_{99} = 9.0542$ is calibrated with $N_{H_0} = 2000$ replicates. The standard error of the q_{99} estimate is $\sqrt{p(1-p)/N}/f(q_{99})$, evaluated at the empirical z-score of q_{99} (2.568), giving 0.017 in $\lambda_{\max}(B)$ units, or 0.186% of its value. The 95% confidence interval is approximately $[9.021, 9.087]$. The corresponding theoretical value, using the standard normal density at $z = 2.326$, is 0.103%. The lowest true positive rate across the five simulated configurations is 0.972 (single clique of size 20), well above the $\alpha = 0.01$ threshold. The qualitative boundary does not depend on the calibration sample size.

Data layer (Section 7). The equilibrium $n^*(B) = \lfloor B/B^* \rfloor$ is exact given the parameters. The attacker profit is -502 with slashing $S_{\text{slash}} = 10$ and $+49.91$ without detection. The discontinuity at $B = B^*$ is a property of the equilibrium, not of the Monte Carlo.

Bootstrap (Section 8). The stationary reputation under net_gain is $R^* = ga/(ga + \mu)$. The measured median $R = 0.3936$ matches the analytic $R^* = 0.4$ for unit activity. The choice $\mu/g = 1.5$ maximizes R_{std} in the sweep; nearby values give similar results.

9.5 Deferred Benchmarks

Two benchmarks are not reported in this paper because they require a live network:

Recursive SNARK proving time. The recursive proof of Section 4.4 composes a Groth16 verifier inside a larger circuit. The proving time depends on the choice of recursion framework and the size of the inner circuit. This is measurable locally but requires implementing the recursive circuit, which is outside the scope of the current paper.

Teleporter latency. The protocol uses Avalanche Warp Messaging (AWM) through the Teleporter contract to propagate reputation commitments across subnets. The message latency depends on the Avalanche consensus timing and is bounded by the finality time of the source subnet. This is not measurable without a live testnet deployment.

Both benchmarks are declared as future work. The protocols for measuring them are defined in the repository.

9.6 Summary of the Section

Claim	Status	Source
Verification suite reproduces all claims	Verified	`verification/`
18 falsifications documented	Verified	`docs/audit/`
`ecPairing` $k = 4 = 181{,}142$ gas	Measured	`GasBenchmarks.sol`
Groth16 verification $\approx$ 206k gas	Measured + estimate	`GasBenchmarks.sol`
Bulletproofs direct: $\sim$6.5M gas	Estimated	`GasBenchmarks.sol`
Recursive SNARK: $\sim$250k gas	Estimated	—
Conclusions stable under parameter variation	Verified	Multiple
Recursive proving time	Deferred	—
Teleporter latency	Deferred	—

The evaluation confirms the reproducibility of the numerical claims and the measured gas costs of the cryptographic primitives. The two deferred benchmarks are declared as future work with the protocols for measuring them.

10 Limitations

10.1 Scope

This section states the limits of the protocol and of the numerical verification. It complements the methodological limitations of Section 9 (validity of numerical methods, sample sizes, discretization bias) with limits that would persist even if the numerical methods were exact. These are limits of the design itself, not of the simulation.

The distinction matters. If a reviewer asks why the paper does not claim to have solved X, the answer is one of two things: either X was not correctly measured (a limitation of Section 9), or X cannot be solved within the design (a limitation of this section).

10.2 Adversary Model

The protocol assumes a rational, profit-maximizing adversary. Every security argument in the paper is conditional on this assumption. Specifically:

The slashing of Section 7 deters fraud only if the adversary prefers profit to loss. A state actor, an ideological opponent, or a competitor with non-monetary utility is not deterred by slashing. The mechanism design does not apply to them.

The cost of Sybil reputation is a market cost. A validator accumulates reputation by producing consensus events on the source chains, and the cost of these events is set by the market for validation on those chains. A Sybil adversary that controls validators on a source chain pays the marginal cost of those validators, not the market cost. The protocol does not model this case; it assumes the adversary does not control the source chains.

The data layer bounty B^* of Section 7 assumes auditors respond to profit. If no verifier is willing to audit at any bounty, the fraud is not detected. The equilibrium $n^*(B) = \lfloor B/B^* \rfloor$ has a solution only if B is at least B^*. Below that threshold, the mechanism provides no detection regardless of the value of the reputation at stake.

These assumptions are standard in the cryptoeconomic literature but they are not universal. The protocol's guarantees do not extend to adversaries that violate them.

10.3 Collusion Detection Boundary

The modularity-spectrum test of Section 5 detects cliques and bipartite structures with high power in the simulated range. The smallest configuration tested (a single clique of size $m = 20$) is detected at 0.972 with $s[-1]$ as the right-tail statistic.

The detection boundary below which the test loses power was not located. Configurations smaller than $m = 20$ were not simulated, and the theoretical Kesten–Stigum threshold for the degree-corrected SBM with the reference parameters ($n = 500$, $\bar{d} = 20$) is below the tested range. The location of the boundary is an open question.

An adaptive adversary who knows the current test may attempt to construct collusion below the (unlocated) boundary. The protocol relies on economic deterrence for this regime rather than on detection.

10.4 Collusion by Bipartite Polarization

The left-tail test measures bipartite polarization via $-\lambda_{\min}(B) > 9.1371$. The power of this test was measured [`verification/rmt/bipartite_power.py`]: true positive rates of 0.990–1.000 across block configurations from balanced (250/250) to heavily unbalanced (50/450), with density ratios $\rho_{\text{between}} \in \{2, 5, 10\}$ and $N_{H_1} = 500$ replicates.

The left-tail test is therefore not a gap. Its power is comparable to the right-tail test. Both tails have been calibrated and measured.

10.5 Reputation Model Selection

The bootstrap analysis of Section 8 uses the net_gain model

$$R' = g \cdot a \cdot (1 - R) - \mu \cdot R \tag{10.1}$$

selected for its correlation between activity and reputation (0.76 vs 0.08–0.11 for the saturating alternatives). The selection does not satisfy all four criteria of the model comparison: the standard deviation of reputation under net_gain is 0.0424 at the sweep optimum, below the 0.05 threshold used in the comparison script. The threshold is a heuristic.

If the true reputation dynamics differ from net_gain — if the activity of a validator has decreasing returns, if reputation decays with a different functional form, if there are non-linear effects — the bootstrap results change. The qualitative conclusion (stake reduction of 20–25% at the median validator) is robust to small changes in the gain rule, but the specific numbers are not.

10.6 Community Structure and Tucker Compression

Section 3 shows that the Tucker decomposition does not capture block structure. A tensor whose entries are constant within communities in mode C has mode-1 and mode-2 unfoldings of full rank, and the truncation to $R_1 = 4$ discards most of the energy. The reconstruction error for a blocky tensor is 52.53%, comparable to that of a random tensor.

If the real reputation tensor has community structure — if validators cluster by chain, by domain, or by region — the Tucker compression is inefficient for that structure. Whether the real tensor is low-rank or blocky is not resolved in this paper. It is an empirical question that requires multi-chain reputation data.

10.7 Calibration on Synthetic Graphs

The RMT calibration of Section 5 uses Erdős–Rényi and Chung–Lu graphs. These are standard null models but they do not capture all features of real collaboration graphs: community hierarchies, temporal correlations between interactions, and directionality.

The empirical thresholds ($q_{99} = 9.0542$ for the right tail, 9.1371 for the left tail) are valid for the calibration distribution. Their validity on real collaboration graphs is an open question. If the real graphs have different spectral properties, the thresholds need recalibration. The recalibration procedure would be the one described in Section 5.4 (Monte Carlo estimation of the null quantiles), applied to real collaboration graphs instead of synthetic ones.

10.8 Bootstrap Assumptions

The bootstrap simulation of Section 8 assumes:

- Activity follows LogNormal$(0, 0.5)$ per validator.
- Validators enter at a rate of 0.03 per epoch.
- The transition rate $\alpha(t)$ is linear in time, reaching 1 at epoch 200.

These are design choices, not empirical facts. The stationary reputation $R^* = ga/(ga+\mu)$ depends on the activity distribution; if real activity is heavier-tailed or lighter-tailed, R^* changes.

The bootstrap numbers (S_f median 0.7764, R median 0.3936, Gini 0.1283) are specific to the reference parameters. The qualitative ranking of policies (hybrid_no_decay dominates hybrid_decay) is robust; the specific numbers are not.

10.9 What the Protocol Does Not Claim

For clarity, the protocol does not claim:

- Security against a non-rational adversary.
- Detection of collusion below the (unlocated) Kesten–Stigum boundary for $s[-1]$.
- Optimality of the Tucker representation for the real reputation tensor.
- Validity of the RMT thresholds on real collaboration graphs.
- Independence of the bootstrap results from the activity distribution.

Each of these is a direction for future work. The protocols for addressing them are defined in the repository.

10.10 Summary of the Section

Limitation	Category	Section
Rational adversary assumption	Design	7, 10.2
Sybil cost is market cost	Design	10.2
Bounty threshold B^*	Design	7
Detection boundary not located	Design	5
Bipartite polarization measured	Verified	5
Net_gain does not satisfy all criteria	Selection	8
Tucker fails for block structure	Method	3
RMT calibrated on synthetic graphs	Method	5
Bootstrap assumes LogNormal activity	Method	8

The methodological limitations (calibration on synthetic data, specific model choices) are separated from the design limitations (non-rational adversary, sub-boundary collusion) because the two require different responses. The methodological limits can be addressed by collecting more data and recalibrating. The design limits require different mechanisms or different claims.

11 Conclusion

11.1 Summary

The Diamond Framework replaces a fraction of the financial stake required of a proof-of-stake validator with verifiable, non-transferable reputation. The protocol is built from five components, each verified numerically in this paper:

1. **Reputation dynamics** (Section 2). The process is a CIR diffusion with multiplicative downward jumps. It remains in its positive domain by construction. The exact mean first-passage time is computed by numerical integration and confirmed by two independent simulation methods.

2. **Tensor representation and compression** (Section 3). The reputation tensor is compressed by Tucker decomposition for storage and transmission. The compression is exact for low-rank tensors and fails for blocky tensors; it is not used in the security path.

3. **Zero-knowledge threshold proof** (Section 4). The original circuit specification is unsound. The protocol adopts a Pedersen commitment with a Bulletproofs inner-product argument, recursively composed for constant on-chain verification cost. The EVM precompile costs are measured locally and match EIP-196 and EIP-197 exactly.

4. **Collusion detection** (Section 5). The modularity matrix $B = W - dd^\top/(2m)$ replaces the normalized adjacency matrix A_{sym}. The empirical null quantile at $\alpha = 0.01$ is 9.0542 with $N_{H_0} = 2000$ replicates. The lowest true positive rate across the five simulated configurations is 0.972; the boundary below which detection degrades is below the tested range. The boundary is characterized as a conjecture.

5. **Payment, data layer, and bootstrap** (Sections 6–8). The payment rule is a saturated uniform posted price, not a Myerson auction. The data layer equilibrium is $n^*(B) = \lfloor B/B^* \rfloor$ with $B^* = c_{\text{audit}}/p_{\text{fraud}}$. The bootstrap under the net_gain reputation model reduces the median financial stake by 22.4%.

11.2 Methodology

Every quantitative claim in this paper is paired with a script that reproduces it. Claims without scripts are labeled as conjectures. The project maintains a public log of falsifications (`docs/audit/numerical-verification.md`) with 18 falsifications: each one a claim that was made during development, tested, and either confirmed or corrected.

The falsification log includes errors of several kinds: a fabricated simulation figure (F-001), a mismeasurement of a gas cost (F-017), a factor of two in an integral (F-011), a rounding rule that shifted a phase transition (F-014), an unverified projection of a multidimensional type (F-004), and an incorrect eigenvalue statistic in the collusion test (F-018). The corrections are documented in the same file as the original claims.

11.3 Contributions

The paper contributes:

1. A cryptoeconomic protocol that uses verifiable reputation as partial collateral, with an explicit adversary model and a stated detection boundary.

2. A numerical verification suite that covers every quantitative claim in the paper, with declared seeds and sample sizes.

3. A falsification log that documents errors of measurement, analysis, and model selection as they were found, with the corrections.

4. Local benchmarks of the EVM precompile costs for the cryptographic primitives used in the protocol.

11.4 Open Problems

The limitations section (Section 10) identifies several open problems that are not resolved in this paper:

- **Non-rational adversaries.** The security arguments assume profit-maximizing adversaries. State actors and ideological opponents are outside the model.

- **Sub-boundary collusion.** The detection boundary below which the modularity-spectrum test loses power was not located. The protocol relies on economic deterrence for this regime.
- **Bipartite polarization.** The left-tail test is calibrated and its power has been measured (0.990–1.000 across tested block configurations).
- **Real reputation data.** The Tucker compression efficiency and the RMT calibration thresholds depend on empirical properties of the reputation tensor and the collaboration graph. Neither is available in this paper.
- **Recursive proving time.** The recursive SNARK of Section 4.4 is designed but not benchmarked.
- **Teleporter latency.** Cross-subnet message propagation requires a live testnet deployment and is not measured.

11.5 Closing Remark

The protocol is in revision. The current document reports the state of the design at the time of writing, with the measurements that have been performed and the claims that remain open. Further development depends on: multi-chain reputation data for calibration, testnet deployment for latency measurement, and external audit for the cryptographic components.

The paper is intended to be readable as a record of what is verified and what is not. The distinction is maintained throughout, and every claim can be traced to a script, a benchmark, or a falsification log entry.

11.6 Summary of the Section

Component	Status	Evidence
Reputation dynamics	Verified	`stochastics/`
Tensor compression	Verified	`tensor/`
ZK threshold proof	Verified	`zk/`, `integrations/avalanche/benchmarks/`
Collusion detection	Verified	`rmt/`
Payment rule	Verified	`mechanism/`
Data layer	Verified	`data-layer/`
Bootstrap	Verified	`bootstrap/`
Reproducibility	Verified	`verification/README.md`
Falsification log	18 entries	`docs/audit/`
External audit	Not performed	—
Real-data calibration	Open	—
Testnet benchmarks	Open	—

The protocol is ready for external audit of the cryptographic components and for testnet deployment of the recursive verification. The remaining open problems are stated in Section 10 and listed in Section 11.

References

[1] Z. Bai and J. W. Silverstein. *Spectral Analysis of Large Dimensional Random Matrices*. Springer Series in Statistics. Springer, 2nd edition, 2010.

[2] F. Bauer and J. Jost. Bipartite and neighborhood graphs and the spectrum of the normalized graph Laplacian. *Communications on Pure and Applied Mathematics*, 2012.

[3] F. Chung and L. Lu. The average distances in random graphs with given expected degrees. *Proceedings of the National Academy of Sciences*, 99(25):15879–15882, 2002.

[4] A. Coja-Oghlan. Graph partitioning via adaptive spectral techniques. *Combinatorics, Probability and Computing*, 19(2):227–284, 2010.

[5] L. W. Cong, Y. Li, and N. Wang. Tokenomics: Dynamic adoption and valuation. *The Review of Financial Studies*, 34(3):1105–1155, 2021.

[6] S. Goldwasser, S. Micali, and C. Rackoff. The knowledge complexity of interactive proof systems. *SIAM Journal on Computing*, 18(1):186–208, 1989.

[7] J. Groth. On the size of pairing-based non-interactive arguments. In *Advances in Cryptology – EUROCRYPT 2016*, pages 305–326. Springer, 2016.

[8] S. D. Kamvar, M. T. Schlosser, and H. Garcia-Molina. The EigenTrust algorithm for reputation management in P2P networks. In *Proceedings of the 12th International Conference on World Wide Web (WWW)*, pages 640–651, 2003.

[9] R. B. Myerson. Optimal auction design. *Mathematics of Operations Research*, 6(1):58–73, 1981.

[10] B. Øksendal and A. Sulem. *Applied Stochastic Control of Jump Diffusions*. Universitext. Springer, Berlin, 2007.

[11] T. Roughgarden. Transaction fee mechanism design. In *Proceedings of the 22nd ACM Conference on Economics and Computation (EC)*, pages 792–792, 2021.

[12] F. Saleh. Blockchain without waste: Proof-of-stake. *The Review of Financial Studies*, 34(3):1156–1190, 2021.

[13] C. A. Tracy and H. Widom. Level-spacing distributions and the Airy kernel. *Communications in Mathematical Physics*, 159(1):151–174, 1994.

[14] L. R. Tucker. Some mathematical notes on three-mode factor analysis. *Psychometrika*, 31(3):279–311, 1966.

[15] E. P. Wigner. On the distribution of the roots of certain symmetric matrices. *Annals of Mathematics*, 67(2):325–327, 1958.

[16] L. Xiong and L. Liu. PeerTrust: Supporting reputation-based trust for peer-to-peer electronic communities. *IEEE Transactions on Knowledge and Data Engineering*, 16(7):843–857, 2004.